

# **AG35-CET&AG35-EUT QuecOpen(SDK)**

## **EAVB 开发指导**

**LTE Standard 模块系列**

版本：1.0.0

日期：2025-05-09

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司  
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233  
电话：+86 21 5108 6236 邮箱：[info@quectel.com](mailto:info@quectel.com)

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：  
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：[support@quectel.com](mailto:support@quectel.com)。

## 前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

## 使用和披露限制

### 许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

### 版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

### 商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

### 第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

## 隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

## 免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2025，保留一切权利。

**Copyright © Quectel Wireless Solutions Co., Ltd. 2025.**

## 文档历史

本文档为临时版本，其中涉及的模块引脚或接口定义、频段、功能、特性及设计等（若有）发生变动的可能性较大，部分参数经过初步验证或仍未验证。变更或补充信息将在后续正式版本中体现。临时版本仅供临时参考使用，若与正式版本存在差异，应以最新版本为准。修订记录如下所示：

### 修订记录

| 版本    | 日期         | 作者         | 变更描述 |
|-------|------------|------------|------|
| -     | 2025-05-09 | Gabriel LI | 文档创建 |
| 1.0.0 | 2025-05-09 | Gabriel LI | 临时版本 |

## 目录

|                                              |           |
|----------------------------------------------|-----------|
| 文档历史 .....                                   | 3         |
| 目录 .....                                     | 4         |
| 表格索引 .....                                   | 5         |
| 图片索引 .....                                   | 6         |
| <b>1 引言 .....</b>                            | <b>7</b>  |
| <b>2 环境要求 .....</b>                          | <b>8</b>  |
| <b>3 Linux PTP 时间同步 .....</b>                | <b>9</b>  |
| 3.1. Linux PTP 简介 .....                      | 9         |
| 3.2. 设置 gPTP 主时钟 .....                       | 9         |
| 3.2.1. 运行 ptp4l .....                        | 9         |
| 3.2.2. 同步 PHC .....                          | 10        |
| 3.3. 设置 gPTP 从时钟 .....                       | 10        |
| 3.3.1. 同步 PHC .....                          | 10        |
| 3.3.2. 同步 Linux 系统时钟 .....                   | 11        |
| 3.4. 确认时钟同步 .....                            | 11        |
| <b>4 EAVB API .....</b>                      | <b>13</b> |
| 4.1. 头文件 .....                               | 13        |
| 4.2. 函数概览 .....                              | 13        |
| 4.3. 函数详解 .....                              | 14        |
| 4.3.1. ql_eavb_init .....                    | 14        |
| 4.3.2. ql_eavb_deinit .....                  | 14        |
| 4.3.3. ql_eavb_gptp_conf_set .....           | 15        |
| 4.3.3.1. ql_eavb_gptp_t .....                | 15        |
| 4.3.3.2. QL_EAVB_GPTP_E .....                | 16        |
| 4.3.4. ql_eavb_gptp_conf_get .....           | 16        |
| 4.3.5. ql_eavb_set_gptp_service_status ..... | 17        |
| 4.3.5.1. QL_EAVB_STATUS_E .....              | 17        |
| 4.3.6. ql_eavb_get_gptp_service_status ..... | 18        |
| 4.4. API 调用流程 .....                          | 18        |
| 4.4.1. 调用接口测试 .....                          | 18        |
| 4.4.2. gPTP 主机端配置 .....                      | 19        |
| 4.4.3. gPTP 从机端配置 .....                      | 20        |
| 4.4.4. 确定 gPTP 同步成功 .....                    | 20        |
| <b>5 附录 参考文档及术语缩写 .....</b>                  | <b>21</b> |

## 表格索引

|                 |    |
|-----------------|----|
| 表 1: 函数概览 ..... | 13 |
| 表 2: 参考文档 ..... | 21 |
| 表 3: 术语缩写 ..... | 21 |

Preliminary  
Quectel Confidential

## 图片索引

|                            |    |
|----------------------------|----|
| 图 1：环境要求.....              | 8  |
| 图 2：使用 tcpdump 抓取的数据包..... | 11 |

Preliminary  
Quectel Confidential

# 1 引言

移远通信 AG35-CET 和 AG35-EUT 模块支持 QuecOpen®方案。QuecOpen®是基于 Linux 的嵌入式开发平台，可简化 IoV 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

gPTP (IEEE Std 802.1AS 协议) 是 IEEE1588 协议的延伸，可以为 TSN 提供全局精确的时钟服务，用以实现 TSN 标准组中的流同步功能。IEEE1588 支持两种路径延时测量方式：End-to-End (E2E) 和 Peer-to-Peer (P2P)，二者不能在同一个网络中共存。在 E2E 机制中，强调的是两个支持 PTP 的端点之间的延时（一个 master，一个 slave），这两个端点可能是直接相连的，也可能中间穿插了普通的交换机、E2E 透明时钟等，在通信双方看来，信息都是在 master 和 slave 直接传输，所以最终 slave 测量到的传输延时是从 master 到 slave 的端到端延时。在 P2P 机制中，要求网络中所有节点必须支持 P2P，所以它强调的是相邻节点间的通信，最终测量的是相邻节点间的传输延时。

E2E 和 P2P 的主要区别如下：

- P2P 测量的是相邻节点间的延时，路径测量报文不会跨节点传输，有利于网络扩展。E2E 测量的是主端口和从端口之间的，中间节点（如 TC、普通 switch）需要转发延时测量报文，网络规模较大时，报文可能泛滥，master 节点压力较大。
- master 节点变更时，E2E 需要重新测量到新 master 节点的路径延时，P2P 只需关心相邻节点。
- E2E 方式允许网络中有普通的 switch（透传 PTP 报文即可，由于驻留时间随机，会影响测量精度），而 P2P 要求网络中的 switch 必须全部支持 P2P。
- E2E 机制中，校时报文和路径测量报文是结合在一起的（典型的 End-to-End 的流程，它使用 Sync、Follow\_Up、Delay\_Req、Delay Resp 四个消息，同时计算时钟偏差和路径测量）。P2P 机制中有独立的报文负责路径测量，将校时和路径测量分开。

本文档适用于 SDK 构建环境的 QuecOpen®方案，主要介绍如何通过开源软件或 QuecOpen SDK 中提供的 EAVB API 运行 gPTP 功能。

## 备注

AG35-CET 和 AG35-EUT 模块仅支持作为从机使用 gPTP 功能，不支持作为主机。但可以作为简单的主机用于调试。

## 2 环境要求

环境要求如下：

- 模块：2 片模块（以 AG35-CET 为例）
- PHY 芯片：2 个（以 YT8521 为例）
- EVB：2 个 EVB
- Linux 内核版本：以移远通信提供的 SDK 为准

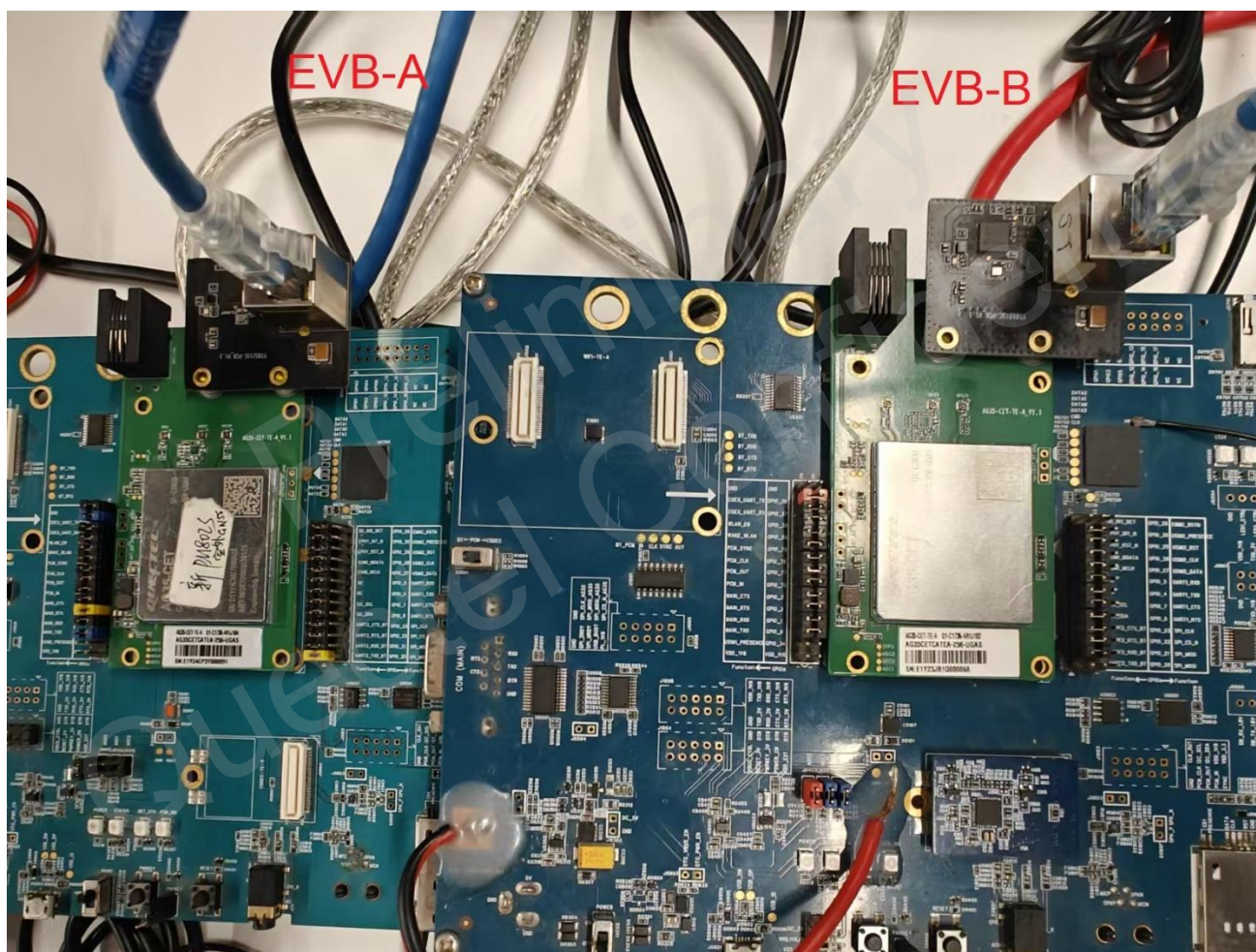


图 1：环境要求

## 3 Linux PTP 时间同步

### 3.1. Linux PTP 简介

Linux PTP 是 Linux 系统下的一种精确时间同步协议的软件，支持 IEEE 1588 PTP 标准，提供了一个用户空间的守护进程 `ptp4l`，用于网络中 PTP 设备通信，实现高精度的时钟同步。另外该软件还包含一个用户程序 `phc2sys`，可以用于 PTP 时钟域和系统时钟之间的同步。Linux PTP 需要和以太网控制器配合，才能最终完成时钟的同步过程。以太网负责记录专用协议包的时间戳信息（源代码位于 `marvell/linux/drivers/net/ethernet/asr/emac_ptp.c`，SYNC、PDELAY\_REQ 和 PDELAY\_RESP 报文需硬件记录时间戳信息），然后发送给用户空间的 Linux PTP 程序。Linux PTP 负责下发协议相关的报文，接收和解析对应的时间戳信息，完成时钟偏差调整和频率同步等工作。时钟同步分为 S0(UNLOCKED)、S1(JUMP)、S2(LOCKED)、S3(LOCKED\_STABLE) 状态。S0 为初始状态，当第一次获取到时钟偏差后，进入 S1 状态，等待调整完成，然后进入 S2 状态进行频率同步，如果连续多次时钟偏差小于某个设置的值，则进入 S3 状态。

### 3.2. 设置 gPTP 主时钟

设置图 1 中 EVB-A 为主时钟（仅供测试，不支持正式使用）。

#### 3.2.1. 运行 ptp4l

在 EVB-A 上运行如下命令，使能主时钟：

```
$ brctl delif bridge0 eth0
$ ifconfig eth0 192.168.1.100 up
$ ptp4l -i eth0 -H -m -f automotive-master.cfg
```

上述命令中，`-i` 选项指定用于控制 `ptp4l` 程序实例的网络接口；`-f` 选项指定的文件（位于 Linux PTP 源码的 `configs` 文件夹下，可联系移远通信技术支持获取）包含 gPTP 模式下运行 `ptp4l` 程序所需的配置项；`-m` 选项使能日志的标准输出。

### 3.2.2. 同步 PHC

由于 EVB-A 为主时钟，故将 PHC 同步到系统时钟，从 ptp4l 程序获取时间偏移量。运行如下命令同步时间：

```
$ phc2sys -s CLOCK_REALTIME -c eth0 -w --transportSpecific=1 --step_threshold=1 -O 0
```

上述命令中，**-s** 选项指定系统时钟的 PHC 为主时钟；**-c** 选项指定 eth0 为从时钟（上述命令中，PHC 管理系统时钟，即系统时钟已调整）；在 gPTP 域运行 phc2sys 程序时需执行 **--transportSpecific**；设置 **--step\_threshold**，表示程序打印“time jumps”时，phc2sys 程序将快速调整时钟时间；**-O** 选项指定 UTC 和 PTP 之间的偏移量。其他参数可通过对应 **help** 命令查询具体用法。

## 3.3. 设置 gPTP 从时钟

设置图1中 EVB-B 为从时钟。

### 3.3.1. 同步 PHC

在 EVB-B 上运行如下命令同步 PHC 为主时钟时间：

```
$ echo 0x0009-0x1A00 > /sys/kernel/debug/eth/phy_reg_dump
$ echo 0x0000-0x9140 > /sys/kernel/debug/eth/phy_reg_dump
$ （若使用 YT8521，可参考如上命令，其他型号请自行参考数据手册）
$ brctl delif bridge0 eth0
$ ifconfig eth0 192.168.1.111 up
$ （此命令执行后需验证两个模块可以相互 ping 通）
$ ptp4l -i eth0 -H -m -f automotive-slave.cfg
```

下图为使用 tcpdump 抓取的数据包：

avb-test.pcap

文件(F) 编辑(E) 视图(V) 跳转(G) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(T) 帮助(H)

eth.type == 0x88f7

| Time      | Source            | Destination    | Protocol | Length | Info                              |
|-----------|-------------------|----------------|----------|--------|-----------------------------------|
| 60.411616 | e4:08:e7:00:00:01 | LLDP_Multicast | PTPv2    | 60     | Sync Message                      |
| 60.411891 | e4:08:e7:00:00:01 | LLDP_Multicast | PTPv2    | 90     | Follow_Up Message                 |
| 60.536904 | e4:08:e7:00:00:01 | LLDP_Multicast | PTPv2    | 60     | Sync Message                      |
| 60.537148 | e4:08:e7:00:00:01 | LLDP_Multicast | PTPv2    | 90     | Follow_Up Message                 |
| 60.655236 | CIMSYS_33:44:55   | LLDP_Multicast | PTPv2    | 68     | Peer_Delay_Req Message            |
| 60.655867 | e4:08:e7:00:00:01 | LLDP_Multicast | PTPv2    | 68     | Peer_Delay_Resp Message           |
| 60.656127 | e4:08:e7:00:00:01 | LLDP_Multicast | PTPv2    | 68     | Peer_Delay_Resp_Follow_Up Message |
| 60.662156 | e4:08:e7:00:00:01 | LLDP_Multicast | PTPv2    | 60     | Sync Message                      |
| 60.662387 | e4:08:e7:00:00:01 | LLDP_Multicast | PTPv2    | 90     | Follow_Up Message                 |
| 60.787487 | e4:08:e7:00:00:01 | LLDP_Multicast | PTPv2    | 60     | Sync Message                      |
| 60.787756 | e4:08:e7:00:00:01 | LLDP_Multicast | PTPv2    | 90     | Follow_Up Message                 |
| 60.912747 | e4:08:e7:00:00:01 | LLDP_Multicast | PTPv2    | 60     | Sync Message                      |
| 60.913005 | e4:08:e7:00:00:01 | LLDP_Multicast | PTPv2    | 90     | Follow_Up Message                 |
| 61.038039 | e4:08:e7:00:00:01 | LLDP_Multicast | PTPv2    | 60     | Sync Message                      |

> Frame 310: 60 bytes on wire (480 bits), 60 bytes captured (480 bits)

▼ Ethernet II, Src: e4:08:e7:00:00:01 (e4:08:e7:00:00:01), Dst: LLDP\_Multicast (01:80:c2:00:00:0e)

> Destination: LLDP\_Multicast (01:80:c2:00:00:0e)

> Source: e4:08:e7:00:00:01 (e4:08:e7:00:00:01)

Type: PTPv2 over Ethernet (IEEE1588) (0x88f7)

Padding: 0000

▼ Precision Time Protocol (IEEE1588)

图 2: 使用 tcpdump 抓取的数据包

### 3.3.2. 同步 Linux 系统时钟

在 EVB-B 上运行以下命令使能 gPTP 从时钟:

```
$ phc2sys -s eth0 -w --transportSpecific=1 --step_threshold=1 -O 0
```

### 3.4. 确认时钟同步

在 ptp4l 程序上, 从时钟上报主时钟计算的时间偏移量。通过该时间偏移量判断系统是否成功同步时钟。ptp4l 程序上报的内容为:

```
root@OpenWrt:/# ptp4l -i eth0 -H -m -f data/automotive-slave.cfg
ptp4l[10924.961]: selected /dev/ptp0 as PTP clock
ptp4l[10925.021]: port 1: INITIALIZING to SLAVE on INIT_COMPLETE
ptp4l[10925.023]: port 0: INITIALIZING to LISTENING on INIT_COMPLETE
ptp4l[10927.127]: rms 847638667583458688 max 1695277335166917888 freq +7095 +/- 2682
delay 1464 +/- 2
ptp4l[10928.228]: rms 42 max 56 freq +8052 +/- 18 delay 1459 +/- 0
```

```
ptp4l[10929.329]: rms 23368 max 45566 freq +30401 +/- 29044 delay -44128 +/- 0
ptp4l[10930.431]: rms 27404 max 45759 freq +15998 +/- 38373 delay 1458 +/- 0
ptp4l[10931.532]: rms 18665 max 30408 freq -19095 +/- 6258 delay 1458 +/- 0
ptp4l[10932.634]: rms 3047 max 4692 freq -351 +/- 4102 delay 1458 +/- 0
ptp4l[10933.736]: rms 5093 max 5360 freq +8839 +/- 1336 delay 1458 +/-
ptp4l[10934.837]: rms 3220 max 4270 freq +10622 +/- 126 delay 1459 +/- 0
ptp4l[10935.939]: rms 1040 max 1703 freq +9635 +/- 370 delay 1459 +/- 0
ptp4l[10937.040]: rms 190 max 317 freq +8528 +/- 240 delay 1458 +/- 0
ptp4l[10938.142]: rms 312 max 336 freq +7991 +/- 84 delay 1458 +/- 0
ptp4l[10939.243]: rms 184 max 255 freq +7911 +/- 29 delay 1459 +/- 0
ptp4l[10940.345]: rms 60 max 93 freq +7968 +/- 24 delay 1459 +/- 0
ptp4l[10940.758]: master offset -15 s3 freq +8003 path delay 1459
ptp4l[10941.858]: master offset 108 s3 freq +8121 path delay 1459
ptp4l[10942.958]: master offset -9 s3 freq +8037 path delay 1459
ptp4l[10944.058]: master offset -11 s3 freq +8032 path delay 1459
ptp4l[10945.158]: master offset 17 s3 freq +8057 path delay 1459
ptp4l[10946.258]: master offset -5 s3 freq +8040 path delay 1459
```

从时钟和主时钟锁定后，ptp4l 程序上报的 **rms** 值会显示 PHC 和主时钟之间的时间偏移量的均方根。若 ptp4l 程序持续上报低于 S3 对应的值，代表主从已经锁定完成，表示 PHC 已同步。

## 4 EAVB API

### 4.1. 头文件

EAVB API 头文件为 `ql_eavb.h`，位于 SDK 的 `ql-sysroots/usr/include/ql-sdk` 路径下。若无特别说明，本文档所述头文件均位于该目录下。此外，结构体定义在头文件 `ql_eavb_common.h` 中，错误码定义在头文件 `ql_type.h` 中。

### 4.2. 函数概览

表 1：函数概览

| 函数                                             | 说明               |
|------------------------------------------------|------------------|
| <code>ql_eavb_init()</code>                    | 创建并初始化 EAVB 设置通道 |
| <code>ql_eavb_deinit()</code>                  | 取消 EAVB 设置通道     |
| <code>ql_eavb_gptp_conf_set()</code>           | 设置 gPTP 协议配置     |
| <code>ql_eavb_gptp_conf_get()</code>           | 获取 gPTP 协议配置     |
| <code>ql_eavb_set_gptp_service_status()</code> | 开启或关闭 gPTP 应用    |
| <code>ql_eavb_get_gptp_service_status()</code> | 获取 gPTP 应用运行状态   |

#### 备注

1. 建议使用开源软件方式运行 gPTP 功能。若不使用开源软件，可使用本章所述 API，但无法调整具体参数。
2. 若无特别说明，本文档所述函数均不支持并发调用，并且不能在相关回调函数中调用以上函数。

## 4.3. 函数详解

### 4.3.1. ql\_eavb\_init

该函数用于创建并初始化 EAVB 设置通道。

- 函数原型

```
int ql_eavb_init(void)
```

- 参数

无

- 返回值

QL\_ERR\_OK

函数执行成功。

QL\_ERR\_UNKNOWN

函数执行失败，未知错误。

QL\_ERR\_SERVICE\_NOT\_READY 函数执行失败，服务器端未准备就绪，需要重试。

其他值

函数执行失败，错误码详见 *ql\_type.h*。

### 4.3.2. ql\_eavb\_deinit

该函数用于取消 EAVB 设置通道。

- 函数原型

```
int ql_eavb_deinit(void)
```

- 参数

无

- 返回值

QL\_ERR\_OK

函数执行成功。

QL\_ERR\_UNKNOWN

函数执行失败，未知错误。

QL\_ERR\_SERVICE\_NOT\_READY 函数执行失败，服务器端未准备就绪，需要重试。

其他值

函数执行失败，错误码详见 *ql\_type.h*。

### 4.3.3. ql\_eavb\_gtp\_conf\_set

该函数用于设置 gPTP 协议配置。

- 函数原型

```
int ql_eavb_gtp_conf_set(ql_eavb_gtp_t *config)
```

- 参数

*config*:

[In] gPTP 协议配置；详见第 4.3.3.1 章。

- 返回值

QL\_ERR\_OK

函数执行成功。

QL\_ERR\_INVALID\_ARG

函数执行失败，无效参数。

QL\_ERR\_NOT\_INIT

函数执行失败，EAVB 设置通道未初始化。

QL\_ERR\_SERVICE\_NOT\_READY 函数执行失败，服务器端未准备就绪，需要重试。

其他值

函数执行失败，错误码详见 *ql\_type.h*。

#### 4.3.3.1. ql\_eavb\_gtp\_t

gPTP 协议配置，结构体定义如下：

```
typedef struct {
    char ifname[20];
    QL_EAVB_GTP_E mode;
    int automotive;
}ql_eavb_gtp_t;
```

- 参数

| 类型            | 参数                | 描述                             |
|---------------|-------------------|--------------------------------|
| char          | <i>ifname</i>     | gPTP 协议网口名称。字符数组；长度：20 字节。     |
| QL_EAVB_GTP_E | <i>mode</i>       | gPTP 运行模式；详见第 4.3.3.2 章。       |
| int           | <i>automotive</i> | 开启或关闭 BMCA 算法。<br>0 开启<br>1 关闭 |

#### 4.3.3.2. QL\_EAVB\_GPTP\_E

gPTP 运行模式，枚举定义如下：

```
typedef enum {
    QL_EAVB_GPTP_SLAVE,
    QL_EAVB_GPTP_MASTER
}QL_EAVB_GPTP_E;
```

##### ● 成员

| 成员                               | 描述                     |
|----------------------------------|------------------------|
| <code>QL_EAVB_GPTP_SLAVE</code>  | gPTP Slave 模式          |
| <code>QL_EAVB_GPTP_MASTER</code> | gPTP Master 模式（仅测试时使用） |

#### 4.3.4. ql\_eavb\_gptp\_conf\_get

该函数用于获取 gPTP 协议配置。

##### ● 函数原型

```
int ql_eavb_gptp_conf_get(ql_eavb_gptp_t *config)
```

##### ● 参数

*config*:

[Out] 获取的 gPTP 协议配置；详见第 4.3.3.1 章。

##### ● 返回值

|                                       |                                       |
|---------------------------------------|---------------------------------------|
| <code>QL_ERR_OK</code>                | 函数执行成功。                               |
| <code>QL_ERR_INVALID_ARG</code>       | 函数执行失败，无效参数。                          |
| <code>QL_ERR_NOT_INIT</code>          | 函数执行失败，EAVB 设置通道未初始化。                 |
| <code>QL_ERR_SERVICE_NOT_READY</code> | 函数执行失败，服务器端未准备就绪，需要重试。                |
| 其他值                                   | 函数执行失败，错误码详见 <code>ql_type.h</code> 。 |

### 4.3.5. ql\_eavb\_set\_gptp\_service\_status

该函数用于开启或关闭 gPTP 应用。设置的网口需要处于启用状态。

#### ● 函数原型

```
int ql_eavb_set_gptp_service_status(QL_EAVB_STATUS_E state)
```

#### ● 参数

*state*:

[In] gPTP 应用运行状态；详见第 4.3.5.1 章。

#### ● 返回值

|                          |                                 |
|--------------------------|---------------------------------|
| QL_ERR_OK                | 函数执行成功。                         |
| QL_ERR_INVALID_ARG       | 函数执行失败，无效参数。                    |
| QL_ERR_NOT_INIT          | 函数执行失败，EAVB 设置通道未初始化。           |
| QL_ERR_SERVICE_NOT_READY | 函数执行失败，服务器端未准备就绪，需要重试。          |
| 其他值                      | 函数执行失败，错误码详见 <i>ql_type.h</i> 。 |

#### 4.3.5.1. QL\_EAVB\_STATUS\_E

EAVB 应用运行状态，枚举定义如下：

```
typedef enum {
    QL_EAVB_STATUS_OFF,
    QL_EAVB_STATUS_ON
}QL_EAVB_STATUS_E;
```

#### ● 成员

| 成员                 | 描述   |
|--------------------|------|
| QL_EAVB_STATUS_OFF | 关闭状态 |
| QL_EAVB_STATUS_ON  | 开启状态 |

#### 4.3.6. ql\_eavb\_get\_gptp\_service\_status

该函数用于获取 gPTP 应用运行状态。

- 函数原型

```
int ql_eavb_get_gptp_service_status(QL_EAVB_STATUS_E *state)
```

- 参数

*state*:

[Out] gPTP 应用运行状态；详见第4.3.5.1章。

- 返回值

|                          |                                 |
|--------------------------|---------------------------------|
| QL_ERR_OK                | 函数执行成功。                         |
| QL_ERR_INVALID_ARG       | 函数执行失败，无效参数。                    |
| QL_ERR_NOT_INIT          | 函数执行失败，EAVB 设置通道未初始化。           |
| QL_ERR_SERVICE_NOT_READY | 函数执行失败，服务器端未准备就绪，需要重试。          |
| 其他值                      | 函数执行失败，错误码详见 <i>ql_type.h</i> 。 |

### 4.4. API 调用流程

#### 4.4.1. 调用接口测试

两个模块分别为 master 和 slave。在终端运行命令 **ql\_sdk\_api\_test**，选择 **15** 即可进入 EAVB API 测试：

```
root@OpenWrt:~# ql_sdk_api_test
0      data call
1      sms
2      nw
3      usb network device
4      voice
5      sim
6      atc
7      dm
8      nf
9      lanhost
10     gnss
11     audio
12     wifi
13     ecall
```

```

14    log
15    eavb
16    data_throughput
-1    exit
Please enter your choice: 15
The current module is:
0    ql_eavb_init
1    ql_eavb_gptp_conf_set
2    ql_eavb_gptp_conf_get
3    ql_eavb_set_gptp_service_status
4    ql_eavb_get_gptp_service_status
5    ql_eavb_deinitg
-1    exit
Please enter your choice:

```

#### 4.4.2. gPTP 主机端配置

依次选择 **0**、**1**、**1**、**1**、**2**，如下图所示：

```

The current module is:
0    ql_eavb_init
1    ql_eavb_gptp_conf_set
2    ql_eavb_gptp_conf_get
3    ql_eavb_set_gptp_service_status
4    ql_eavb_get_gptp_service_status
5    ql_eavb_deinitg
-1    exit
Please enter your choice: 0

->Item : ql_eavb_init
Start to ql_eavb_init
Successful
Please enter your choice:

The current module is:
0    ql_eavb_init
1    ql_eavb_gptp_conf_set
2    ql_eavb_gptp_conf_get
3    ql_eavb_set_gptp_service_status
4    ql_eavb_get_gptp_service_status
5    ql_eavb_deinitg
-1    exit
Please enter your choice: 1

->Item : ql_eavb_gptp_conf_set
please input gptp (master/slave)(1/0): 1
please input gptp enable(1/0): 1
Successful
Please enter your choice: gptp_config.ifname = eth0  gptp_config.mode = 1  gptp_config.automotive = 1

The current module is:
0    ql_eavb_init
1    ql_eavb_gptp_conf_set
2    ql_eavb_gptp_conf_get
3    ql_eavb_set_gptp_service_status
4    ql_eavb_get_gptp_service_status
5    ql_eavb_deinitg
-1    exit
Please enter your choice: 2

->Item : ql_eavb_gptp_conf_get
Successful
ifname = eth0, mode = 1(0/1), automotive = 1
Please enter your choice:

```

#### 4.4.3. gPTP 从机端配置

依次选择 0、1、0、1、2，如下图所示：

```

1      exit
Please enter your choice: 15

The current module is:
0      ql_eavb_init
1      ql_eavb_gptp_conf_set
2      ql_eavb_gptp_conf_get
3      ql_eavb_set_gptp_service_status
4      ql_eavb_get_gptp_service_status
5      ql_eavb_deinitg
-1     exit
Please enter your choice: 0

->Item : ql_eavb_init
Start to ql_eavb_init
Successful
Please enter your choice: 1

->Item : ql_eavb_gptp_conf_set
please input gptp (master/slave)(1/0): 0
please input gptp enable(1/0): 1
Successful
Please enter your choice: 2

->Item : ql_eavb_gptp_conf_get
Successful
ifname = eth0, mode = 0(0/1), automotive = 1
Please enter your choice: 3

```

#### 4.4.4. 确定 gPTP 同步成功

从机执行 **logcat -s ql\_eavbd**，打印如下日志即代表同步成功：

```

I/ql_eavbd(1797): [update_clock_643]:CLOCK_REALTIME phc offset -4546 s2 freq -14654 delay 0
I/ql_eavbd(1797): [clock_synchronize_1777]:master offset -48 s3 freq -6405 path delay 1959
I/ql_eavbd(1797): [update_clock_643]:CLOCK_REALTIME phc offset 4948 s2 freq -6524 delay 0
I/ql_eavbd(1797): [clock_synchronize_1777]:master offset 43 s3 freq -6328 path delay 1959
I/ql_eavbd(1797): [update_clock_643]:CLOCK_REALTIME phc offset 3671 s2 freq -6316 delay 0
D/ql_eavbd(1797): [tsproc_update_delay_161]:delay filtered 1957 raw 1955
I/ql_eavbd(1797): [clock_synchronize_1777]:master offset 51 s3 freq -6307 path delay 1957
I/ql_eavbd(1797): [update_clock_643]:CLOCK_REALTIME phc offset -6551 s2 freq -15437 delay 0
I/ql_eavbd(1797): [clock_synchronize_1777]:master offset 56 s3 freq -6287 path delay 1957
I/ql_eavbd(1797): [update_clock_643]:CLOCK_REALTIME phc offset 5170 s2 freq -5681 delay 0
I/ql_eavbd(1797): [clock_synchronize_1777]:master offset -53 s3 freq -6379 path delay 1957
I/ql_eavbd(1797): [update_clock_643]:CLOCK_REALTIME phc offset -3823 s2 freq -13123 delay 0
I/ql_eavbd(1797): [clock_synchronize_1777]:master offset -47 s3 freq -6389 path delay 1957
I/ql_eavbd(1797): [update_clock_643]:CLOCK_REALTIME phc offset 4192 s2 freq -6255 delay 0
D/ql_eavbd(1797): [tsproc_update_delay_161]:delay filtered 1957 raw 1956
I/ql_eavbd(1797): [clock_synchronize_1777]:master offset -42 s3 freq -6398 path delay 1957
I/ql_eavbd(1797): [update_clock_643]:CLOCK_REALTIME phc offset 736 s2 freq -8454 delay 0
I/ql_eavbd(1797): [clock_synchronize_1777]:master offset 49 s3 freq -6320 path delay 1957

```

## 5 附录 参考文档及术语缩写

表 2: 参考文档

| 文档名称                                               |
|----------------------------------------------------|
| [1] Quectel_AG35-CET&AG35-EUT_QuecOpen(SDK)_快速开发指导 |

表 3: 术语缩写

| 缩写   | 英文全称                                              | 中文全称      |
|------|---------------------------------------------------|-----------|
| BMCA | Best Master Clock Algorithm                       | 最佳主时钟算法   |
| EAVB | Ethernet Audio Video Bridging                     | 以太网音视频桥接  |
| EVB  | Evaluation Board                                  | 评估板       |
| gPTP | Generalized Precision Time Protocol               | 广义精确时间协议  |
| IEEE | Institute of Electrical and Electronics Engineers | 电气电子工程师学会 |
| IoV  | Internet of Vehicles                              | 车联网       |
| PHC  | PTP Hardware Clock                                | PTP 硬件时钟  |
| PHY  | Physical                                          | 端口物理层     |
| PTP  | Precision Time Protocol                           | 精确时间协议    |
| SDK  | Software Development Kit                          | 软件开发工具包   |
| TC   | Traffic Control                                   | 流量控制      |
| TCP  | Transmission Control Protocol                     | 传输控制协议    |
| TSN  | Time-Sensitive Networking                         | 时间敏感网络    |
| UTC  | Coordinated Universal Time                        | 协调世界时     |